

Transformer — Three Core Concepts

A descriptive set of notes covering (1) ANN/FFN, (2) Layer Normalization with Pre-Norm vs Post-Norm, and (3) why Transformers learn separate Q, K and V projections.

1. Why is ANN / FFN used in Transformers?

ANN here usually refers to the Feed-Forward Neural Network (FFN) inside each Transformer block. Attention and FFN perform different jobs. Attention allows tokens to communicate with one another and collect contextual information. The FFN then processes the representation of each token independently, adding nonlinear transformation and feature-extraction capacity.

The key distinction

Attention = “What information should I look at?”

Attention mixes information across different token positions.

FFN/ANN = “What should I do with the information I received?”

The FFN transforms the contextual representation of each token independently.

Typical FFN computation

$$\text{FFN}(x) = W \cdot \text{Activation}(Wx + b) + b$$

A common pattern is: model dimension $\rightarrow$ larger hidden dimension $\rightarrow$ model dimension. For example, a 512-dimensional representation may be expanded to 2048 dimensions, passed through GELU/ReLU, and projected back to 512 dimensions. The exact dimensions depend on the model.

Where FFN appears

Encoder: Self-Attention $\rightarrow$ Add/Norm $\rightarrow$ FFN $\rightarrow$ Add/Norm.

Decoder: Masked Self-Attention $\rightarrow$ Add/Norm $\rightarrow$ Cross-Attention $\rightarrow$ Add/Norm $\rightarrow$ FFN $\rightarrow$ Add/Norm.

Important mental model

Attention communicates between tokens; FFN processes each token after that communication.

2. Layer Normalization, Pre-Norm and Post-Norm

Layer Normalization (LayerNorm) helps keep the feature values of each token representation in a well-behaved range, making optimization more stable. Unlike BatchNorm, LayerNorm normalizes across the features of an individual token rather than relying on batch statistics, which fits sequence models well.

LayerNorm concept

$$\text{LN}(x) = \gamma \cdot (x - \mu) / \sqrt{(\sigma^2 + \epsilon)} + \beta$$

Here μ and σ^2 are computed from the features of the representation being normalized. γ and β are learnable parameters, allowing the network to adjust the normalized representation.

Residual connection

A Transformer sublayer uses a residual/skip connection so the original input can be added back:

$$\text{Output} = x + \text{Sublayer}(x)$$

Post-Norm

Post-Norm places LayerNorm after the residual addition:

$$\text{Output} = \text{LayerNorm}(x + \text{Sublayer}(x))$$

Conceptually: $x \rightarrow \text{Sublayer} \rightarrow \text{Add residual} \rightarrow \text{LayerNorm}$.

Pre-Norm

Pre-Norm places LayerNorm before the sublayer:

$$\text{Output} = x + \text{Sublayer}(\text{LayerNorm}(x))$$

Conceptually: $x \rightarrow \text{LayerNorm} \rightarrow \text{Sublayer} \rightarrow \text{Add residual}$.

Why Pre-Norm is often preferred for deep Transformers

In Pre-Norm, the residual stream contains a relatively direct path from one layer to the next. This gives gradients and information an easier route through many layers and generally makes deep Transformer optimization more stable.

Post-Norm can also work very well, but its normalization sits after the residual addition, so optimization can be more sensitive in very deep networks.

Memory trick

Pre-Norm: Normalize before the sublayer.

Post-Norm: Normalize after the residual addition.

LayerNorm: Stabilizes representations.

Residual: Preserves an easier information/gradient path.

3. Why multiply by WQ, WK and WV?

For an input representation X, self-attention creates three different representations:

$$Q = XW_Q \mid K = XW_K \mid V = XW_V$$

The matrices WQ, WK and WV are learned parameters. They let the model transform the same token embedding into three different spaces for three different purposes.

The three jobs

Representation	Role	Intuition
Query (Q)	Represents what this token is looking for	What information do I need?
Key (K)	Represents what a token can be matched with	What kind of information do I contain?
Value (V)	Represents information that will actually be passed forward	What information should I provide?

Why not simply use X directly as Q, K and V?

It is theoretically possible to set $Q = X$, $K = X$ and $V = X$, but this removes the model's ability to learn three independent views of the same embedding. Q and K need to be useful for determining relevance, while V needs to carry the information that should be transferred. Separate learned projections give the model this flexibility.

How Q, K and V work together

First, Q and K are compared to determine how strongly one token should attend to another:

$$\text{Attention scores} = \text{Softmax}(QK^T / \sqrt{d_K})$$

These scores become attention weights. The weights are then applied to V:

$$\text{Attention output} = \text{Attention weights} \times V$$

The strongest mental model

Q and K determine WHERE to look; V determines WHAT information to take.

Putting all three concepts together

A useful end-to-end mental model for a Transformer block is:

1. Q/K/V projections create specialized representations for attention.
2. Attention lets tokens communicate and gather contextual information.
3. Residual connections preserve the original representation and support gradient flow.
4. LayerNorm stabilizes the representations (before or after the sublayer depending on architecture).
5. The FFN/ANN nonlinearly processes each token's contextual representation.

One-line summary

Q/K/V = specialize the information for attention; Attention = communicate; LayerNorm = stabilize; Residual = preserve/route information; FFN/ANN = transform and enrich features.